Oracle Observability of Quantum Transpiler Regressions: What Output-Equivalence Oracles Miss

A Cross-SDK Mining Study and a Fault-Class-Matched Oracle Family for the Output-Invisible Channels (Qiskit, tket, Cirq)

Furqan Nasir[1,2], Arif Shah[1], Iftikhar Alam[1]

[1] City University of Science and Information Technology (CUSIT), Peshawar, Pakistan (PhD Student, Computer Science)

[2] Department of Computer Science, National University of Computer and Emerging Sciences (FAST-NUCES), Islamabad, Pakistan (Lecturer)

Corresponding author: Furqan Nasir (furqannr@gmail.com) · ORCID: Furqan Nasir 0000-0002-5259-3448, Arif Shah 0000-0003-0090-3333

Abstract

Quantum SDKs such as Qiskit are revised continually, and a single transpiler-pass change can introduce a regression. The field's de-facto correctness criterion is output equivalence: compare the compiled circuit's output map modulo global phase and qubit-layout permutation. We show this criterion is systematically incomplete. A repository-mining study of 68 merged Qiskit transpiler bug-fixes — independently human-coded by multiple raters (three on a 24-fix adjudicated seed, two on the remaining 44; Cohen's $\kappa = 0.67$ and 0.86, adjudicated against a frozen codebook) — finds that about 28% (95% Wilson CI 19–40%) lie in channels an output-equivalence oracle cannot observe: corrupted layout/permutation contract metadata, non-determinism, and dropped global phase. The gap is not Qiskit-specific: it replicates on two other compilers (tket 33%, Cirq 20%, bracketing Qiskit's 28%), dominated everywhere by the layout/permutation-metadata channel. These output-invisible fixes are statistically indistinguishable from the observable ones in patch size and merge latency (Mann–Whitney U, all $p > 0.1$), so the gap is not explained by fix triviality. We make the gap concrete by building the fault-class-matched oracle family that closes it — a layout/permutation contract differ, contract-level metamorphic relations (MR-1, permutation consistency), and a global-phase-tracking oracle, all layout-aware and width-tiered to avoid full unitaries beyond small circuits. We then verify three of the mined output-invisible fixes directly from source, across two channels — the contract/metadata fixes ElidePermutations #14603 and final_layout composition #14919, and a global-phase fix in CommutativeCancellation #14956 — on which the output oracle is provably blind while the matched oracle detects the fault. A held-out evaluation on mutation-engine faults finds the oracle family channel-specific (no false positives in 54 runs) and sensitive to a held-out phase corruption. The whole study rests on a leakage-safe evaluation methodology — non-pooled event cohorts, strict provenance control, six implementation-validity gates, and a claim-scope rule — so that the observability and detection claims are auditable and reproducible. All code, data, and artifacts are released.

Keywords: Quantum software engineering · Quantum transpiler · Test-oracle problem · Quantum circuit equivalence checking · Metamorphic testing · Qiskit

1. Introduction

Quantum software development kits (SDKs) such as Qiskit are developed at a rapid and continuous pace, and the compilers inside them — the transpilers that map an abstract circuit onto a hardware-native gate set and a limited qubit connectivity — are revised very often. A single change in a layout, routing, translation, optimization, or scheduling pass can introduce a regression, and empirical studies confirm that compiler and transpiler components are an important source of defects in quantum platforms [1]. How does one know a transpiler change is correct? The field's de-facto answer is an output-equivalence oracle: because transpilation may permute qubits and shift a global phase, a compiled circuit is accepted when its output map equals the source's modulo global phase and modulo a qubit-layout permutation. This criterion underlies quantum circuit equivalence checking and the equivalence oracles used across quantum benchmark and testing suites.

Our starting observation is that this criterion is systematically incomplete. A transpiler pass can corrupt the recorded layout/permutation contract (the TranspileLayout, final_index_layout, and routing_permutation metadata), make compilation non-deterministic across fixed seeds, or drop the tracked global phase, while leaving the layout-applied output map correct. Such a fault is invisible to an output-equivalence oracle by construction: the compiled program is “right”, yet the recorded metadata a downstream consumer relies on is wrong. This is not hypothetical — a real Qiskit regression in ElidePermutations (#14603) corrupted the recorded permutation while the applied output stayed correct, and passed the output oracle. We therefore ask how common such output-invisible faults are, whether the phenomenon is specific to Qiskit, and what it takes to detect them.

This paper’s central contribution is an observability result about quantum transpiler testing, the fault-class-matched oracle family that acts on it, and the leakage-safe methodology that frames it. Concretely:

1. **An oracle-observability result.** The de-facto correctness criterion for quantum transpilers — output equivalence modulo global phase and qubit permutation — is systematically incomplete. A repository-mining study of 68 merged Qiskit transpiler bug-fixes, independently human-coded by multiple raters (three on a 24-fix adjudicated seed, two on the remaining 44; Cohen's κ = 0.67 and 0.86, adjudicated against a frozen codebook), finds that roughly 28% (95% Wilson CI 19–40%) fall in output-invisible channels: contract/metadata, non-determinism, and global phase. We verify three of these fixes directly from source, spanning two channels (contract/metadata: ElidePermutations #14603 and final_layout composition #14919; global phase: CommutativeCancellation #14956): the output oracle passes the buggy build while the fault is real, reachable, and caught by a matched oracle.
2. **A fault-class-matched oracle family that closes the gap.** We build and validate three oracles, one per output-invisible channel: a layout/permutation contract differ; contract-level metamorphic relations (MR-1, permutation consistency — a new oracle class for transpiler metadata); and a global-phase-tracking oracle, the exact complement of output equivalence. Each is layout-aware and width-tiered (exact operator for n ≤ 12 qubits, sampled layout-normalized statevectors for 13–22, structural beyond), never forming a full 2^n x 2^n unitary at scale. With the layout method fixed the differ is well-calibrated, and under the default layout path it auto-surfaces a fixed-seed layout-metadata non-determinism (present across Qiskit 2.4.2 and 2.5.0).
3. **A leakage-safe evaluation methodology.** We separate change events into non-pooled cohorts (forward regression, fix-boundary differential), reproduce historical regressions from source, apply tiered quantum-aware oracles with an empirical false-positive rate, and enforce six implementation-validity gates and a strict claim-scope rule, so that every observability and detection claim rests on auditable, leakage-safe, reproducible evidence.

We answer two research questions. RQ1 (observability): how often do real, merged transpiler bug-fixes fall in channels a black-box output-equivalence oracle cannot observe, does this rate replicate across compilers, and are the output-invisible fixes distinguishable from the observable ones by surface characteristics such as patch size or merge latency? RQ2 (detection): can a fault-class-matched oracle family — a layout/permutation contract differ, contract-level metamorphic relations, and a global-phase tracker — detect these output-invisible faults, verified from source on real regressions? Section 2 gives background, Section 3 surveys related work, Section 4 presents the methodology, Section 5 the implementation, and Section 6 the evaluation. Sections 7–9 discuss findings, threats, and future work, Section 10 concludes, and Section 11 is the artifact statement.

2. Background

2.1 The Qiskit transpiler

Qiskit compiles an abstract circuit to a target backend through a staged pass manager [7]. The canonical stages are layout, routing, translation, optimization, and scheduling, bundled at optimization levels 0–3. Two properties matter for testing: the actually executed passes depend on the circuit, backend, configuration, and the exact Qiskit revision (so they must be observed, not

inferred from source); and transpilation may permute qubits, so a transpiled circuit equals the original only modulo a layout permutation.

2.2 Transpiler regression types and their manifestation

We distinguish what kind of defect a regression is from how, and indeed whether, it shows up in an observable signal, because the gap between these two is the central finding of this study. The fault-type grouping itself is conventional. The correctness group contains functional faults (a compilation failure) and semantic faults (a different unitary or output). The quality group covers circuit-quality faults, such as a worse two-qubit count or a larger depth. The performance group covers compilation-performance faults. These groupings are not mechanical: a crash is not automatically a semantic fault, and an increase in two-qubit count is not a quality regression if the depth improves substantially. For that reason we judge quality with a Pareto rule over pre-declared thresholds and report the two metrics separately.

Crucially, two regressions of the same fault type can manifest in very different observable signals, and some manifest in signals a black-box, output-only oracle never inspects. We therefore use a fault-manifestation taxonomy, the channel through which a regression could be detected:

Table 1. Study-specific fault-manifestation taxonomy (detection channel).

Manifestation channel	Observable signal	Output oracle sees it?
output_semantic	different unitary/output state	Yes (within oracle width)
compilation_failure	crash, exception, invalid circuit	Yes
circuit_quality	worse two-qubit count or depth	Yes
performance	slower / more memory-hungry compilation	Yes, if over the screen
transpiler_contract_or_metadata	wrong internal property (TranspileLayout, final_layout, routing_permutation) with applied output still correct	No, invisible to output oracles
determinism_or_reproducibility	output varies across fixed-seed runs, often only under a specific (e.g. noisy) target	Only with a determinism oracle on that target

This taxonomy is what makes the historical results in Section 6.2 interpretable, since a regression can be entirely real and yet sit in a channel that our black-box pipeline never observes. The ElidePermutations regression (H1), for instance, manifests in the transpiler_contract_or_metadata channel: it corrupts a layout property rather than producing a wrong output unitary. The VF2Layout regression (H3) manifests as determinism_or_reproducibility under a noisy target, which is neither a circuit_quality nor an output_semantic signal. In both cases a “pass” from the output oracle is exactly what one should expect, and it is not evidence that the oracle under-detects quality or semantic faults.

2.3 The oracle problem for transpilation

Deciding whether a transpiled circuit is correct is an instance of the quantum oracle problem [8], which is the central obstacle catalogued in the general test-oracle survey [9]. When a circuit is small and unitary-pure, one can normalize the transpiled layout and then compare unitaries modulo global phase. This comparison is exact, but it does not scale, because an n-qubit operator already has 2^n × 2^n complex entries. Larger or non-unitary circuits must instead be checked with sampled-statevector, observable, metamorphic, or structural methods [3]. One consequence is central to our findings: a black-box oracle only observes the output, so a fault that corrupts internal metadata while leaving the output unitary intact can remain invisible to it. Dedicated quantum equivalence-checking methods exploit reversibility and simulation to compare a circuit before and after compilation efficiently [10], yet they too reason about the output map and not about a transpiler's internal contracts.

3. Related Work

We position the work relative to quantum software and compiler testing, and close with an explicit comparison.

3.1 Quantum software and compiler testing

Our survey of quantum software and compiler testing follows a systematic review conducted to a pre-registered, PRISMA-based protocol (included in the artifact), from which the primary studies discussed here are drawn. Quantum software testing must also contend with the oracle problem [8]. Metamorphic and differential testing are two ways to mitigate it. MorphQ, for example, generates diverse programs and applies quantum-specific metamorphic relations to test Qiskit, and in doing so exposes real bugs [3], while MorphQ++ reproduces and extends metamorphic testing of quantum compilers [4]. Quantum software engineering is by now an established research thread, including within this venue [11], where the transpiler itself is an active optimization target [12]. Several recent frameworks sharpen the picture. QuCheck offers property-based testing of quantum programs in Qiskit, with flexible generators, assertions, and preconditions that are evaluated through mutation analysis [5]. QEMI adapts equivalence-modulo-inputs compiler testing to quantum stacks, generating program variants by removing dead code and uncovering crash and behavioural bugs across Qiskit, Q#, and Cirq [6]. QDiff applies differential testing to quantum software stacks, exploring semantics-preserving variants and filtering circuits by static characteristics to compare Qiskit, Cirq, and Pyquil [13], and QSPE enumerates skeletal program variants with statevector-based validation, reporting 81 Qiskit miscompilations that developers have acknowledged [14]. Concolic testing targets transpiler decision points, parameter thresholds, and layout heuristics, surfacing missed optimizations and semantic deviations in Qiskit passes [15]. A complementary, formal line of work verifies the compiler directly: Giallar provides push-button verification of Qiskit passes, verifying 44 of 56 passes across 13 versions and uncovering three bugs [16]. Static analysis and bug benchmarks round out the landscape, with LintQ detecting quantum-specific defects that general linters miss [17] and Bugs4Q curating real, reproducible Qiskit bugs [18]. Mutation analysis has likewise been adapted to quantum programs in Muskit [19] and QMutPy [20], and a recent roadmap explicitly names transpilation as an under-studied testing target [21]. Empirical studies confirm that compiler and transpiler components are a significant source of defects [1], and Benchpress benchmarks circuit construction, manipulation, and compilation across SDKs [2]. All of these efforts aim at bug finding, test generation, or verification. Crucially, they judge correctness by output equivalence, and none asks whether that criterion itself misses a class of transpiler fault, which is the question this paper takes up.

3.2 Positioning

The threads above share an implicit assumption: that a compiled circuit is correct when it is output-equivalent to the source, modulo global phase and qubit permutation. Equivalence checking takes this as the correctness criterion; metamorphic and differential testing (MorphQ [3], QDiff [13], QEMI [6], QSPE [14]) generate program variants and compare their outputs under it; even direct verification (Giallar [16]) certifies input–output behaviour. Our contribution is orthogonal to all of these: we show that the output-equivalence criterion is systematically incomplete. A substantial minority of real transpiler regressions corrupt layout/permutation metadata, determinism, or global phase while leaving the output map correct, and we build the fault-class-matched oracle family that observes these channels, verified from source and replicated across compilers. In doing so, contract-level metamorphic relations (MR-1, permutation consistency) extend program-level metamorphic testing to transpiler metadata, an open direction that the prior threads (equivalence checking, metamorphic and differential testing, and direct verification) do not address. Open QBench [22] benchmarks platform performance, which is complementary to observing a transpiler's internal contracts.

4. Methodology

4.1 Problem formulation

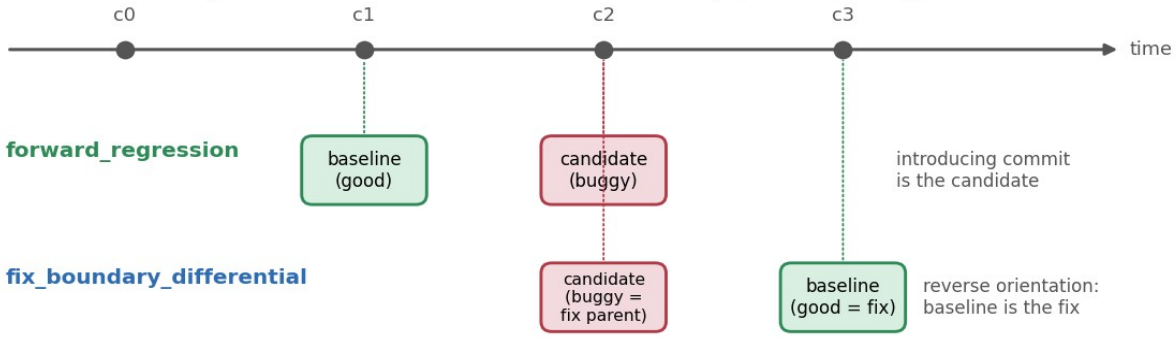
Given a transpiler change, we ask two things. First, is a regression it introduces observable at all to the reference correctness screen, meaning output equivalence modulo global phase and qubit

permutation, augmented with compilation-validity, circuit-quality, and performance checks? Second, when it is not, can a fault-class-matched oracle for the corresponding channel detect it from source? The primary evaluation unit is the (event_id, test_id) pair, and every regression is classified by the manifestation channel through which it could be detected (§2.2).

4.2 Change-event model and cohorts

The unit of the dataset is a change event, not a commit: an event is a tuple (baseline_revision, candidate_revision, change_metadata, fault_id), each carrying a unique fault_id and a fault_type. Every event declares one of three cohorts, and the cohorts are reported separately and never pooled. A forward_regression event pairs the last known-good parent (the baseline) with the regression-inducing commit (the candidate), and it is the only cohort that models the real CI question. A fix_boundary_differential event is used in reverse orientation when the introducing commit has not been traced: it pairs a fix commit (the baseline) with its parent (the candidate), which yields a fault-revealing differential that we never describe as a prediction of an incoming change. A mutation event injects a single controlled fault on a base revision and is used only to fill gaps in fault-type coverage. Throughout, historical events are preferred.

Figure 2. Cohort orientations on a commit timeline (reported separately).



4.3 Test units, manifest, and provenance

A test unit is a tuple (circuit, backend, transpiler_configuration, oracle). A static manifest stores its metadata, while a separate calibrated profile records the measured baseline cost, the pass stages that actually executed (obtained through instrumentation rather than inferred from filenames), routing observations, and peak memory, keyed by (event_id, baseline_sha, test_id, event_environment_id). Each unit also declares its provenance, one of pre_existing, extracted_from_fix, or synthesized, together with an available_before_candidate flag, and these support the two evaluations described in §4.5.

4.4 Oracles

Semantic. The semantic oracle is tiered by applicability and width. For a small circuit (at most 12 qubits) that is unitary-pure and ancilla-free, it checks exact unitary equivalence modulo global phase after first normalizing the layout. For every other circuit it falls back to structural validity, meaning adherence to the basis and to the coupling map. A full 2^n operator is therefore never materialized for large circuits. This mirrors dedicated quantum equivalence-checking practice, which compares the maps of the pre- and post-compilation circuits and exploits reversibility and simulation to stay tractable [10].

Quality. A Pareto rule over pre-declared thresholds; Δ two-qubit count and Δ depth reported separately; a sensitivity grid is reported and thresholds are fixed before held-out evaluation.

Performance. A two-stage protocol (median-slowdown screen, then robust effect size); exploratory on a laptop, timed with CPU process time.

Empirical false-positive rate. A warning is a false positive only if not upheld by independent confirmation, never merely because an oracle fired.

4.5 Anti-leakage controls

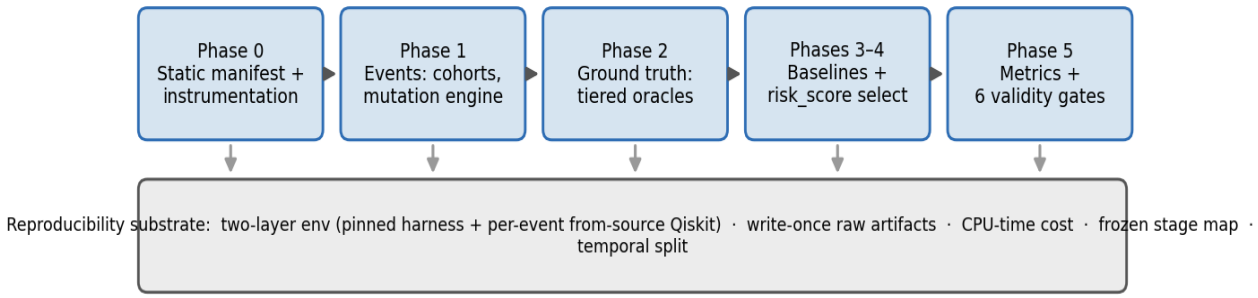
Four controls enforce leakage safety. The first is a group-level temporal split keyed by split_group_id = base_revision + mutation_family, which sends any group that straddles the cutoff entirely to the test side. The second is provenance-driven: a Primary CI evaluation restricts the candidates to tests that were available before the candidate, whereas post-fix targeted triggers are confined to a Secondary retrospective evaluation. The third freezes the change–stage map before held-out evaluation, and any later refinement is performed on training events only and is versioned. The fourth calibrates cost with CPU process time (time.process_time) and retains wall-clock time only as a secondary reference.

4.6 Validity gates and claim scope

Six gates check implementation correctness before any result is reported: budget compliance, reproducibility, absence of leakage, label reproducibility from the write-once artifacts, metric correctness against hand-computed cases, and oracle coverage. A claim-scope rule then forbids any headline temporal-generalization claim while fewer than three forward_regression events are verified in the held-out cohort, and in that situation results are reported per event.

5. Implementation

Figure 1. The cart pipeline (Phases 0–5) over a shared reproducibility substrate.



The methodology is realized in a Python prototype, cart, organized by phase (manifest, events, oracles, labels, selectors, metrics, gates) with a CLI (manifest, events, ground-truth, select, evaluate, historical-run).

Two-layer environment. Reproducing historical regressions means building several different Qiskit revisions, which is incompatible with a single pinned Qiskit. We therefore separate a fixed harness layer (Python 3.11, Benchpress, and the oracle and measurement stack, all lockfile-pinned) from a Qiskit-under-test layer that is built from source for each event. Instrumentation is anchored on Qiskit 2.4.2 within the 2.x window. Building from source requires a Rust toolchain.

Per-event from-source runner. For historical events, a runner builds the baseline and candidate revisions into isolated virtual environments. In each environment a worker runs as a subprocess, transpiles the test units with its own Qiskit, and serializes the outputs (as QPY) together with the CPU timings. The harness then loads both sets of results back and applies the shared oracle pipeline. In this way the system under test stays isolated, while a single oracle implementation is reused for both revisions.

Reproducibility artifacts. Raw oracle evidence is written once and never overwritten, and the derived labels are regenerated from it so that they reproduce exactly, including the timing-dependent fields. The per-event environment recipes, the seeds, the pre-declared thresholds, the frozen stage map, and the frozen per-run configurations are all committed. The prototype also ships an automated test suite, with the six validity gates wired directly into evaluate.

What is implemented versus exercised. To avoid over-stating coverage we separate what is implemented and exercised from what is deferred.

- **Implemented and exercised:** the per-event from-source runner; the fault-class-matched oracle family — the layout/permutation contract differ, the metamorphic MR-1 (permutation-consistency) oracle, and the global-phase tracker — and the source verification of H1 (ElidePermutations #14603), H2 (final_layout composition #14919), and the CommutativeCancellation global-phase fix #14956, on each of which the black-box output oracle is blind while the matched oracle detects the fault; the targeted-trigger units and the Secondary retrospective evaluation; and the exploratory performance Stage-1 screen. The determinism (#14730) event remains unreproduced at this scale; the small-overhead performance event (#14120, H4) is confirmed as a forward-regression (§6.2).
- **Deferred:** a noisy-target determinism oracle; a broader sampled-statevector campaign at scale (13–22 qubits); and a controlled-hardware, three-run-median performance protocol.

6. Evaluation

We evaluate the two research questions on a single 16 GB laptop, anchored on Qiskit 2.4.2, using real historical regressions and the mined corpus.

6.1 Corpus and setup

The corpus centres on real historical Qiskit transpiler regressions identified from fix pull requests, each with exact candidate and baseline commit SHAs (Table 2), and on the mined corpus of merged transpiler bug-fixes analysed in §6.3. Historical events require per-event from-source builds of the baseline and candidate revisions and are reported in the audit (Table 3). To stay within a single 16 GB laptop, trigger circuits are width-capped at 8 qubits for the exact-unitary oracle tier, and the fault-class-matched oracles fall back to sampled statevectors (13–22 qubits) or structural checks beyond that. Thresholds, seeds, and the frozen change–stage map are pre-declared before evaluation. Controlled mutation is an established way to construct faulty quantum-circuit benchmarks with characterizable detectability [23], and our reporting follows recent guidance on empirical quantum-software-testing studies [24].

Table 2. Audited event ledger and reader's guide to the historical events H1–H5 (rows ordered H1–H5; cohorts never pooled).

Event(s)	Cohort	Fault type	Status
H1 ElidePermutations (#14603)	fix_boundary	contract/metadata	source-verified; output-invisible
H2 final_layout composition (#14919)	fix_boundary	contract/metadata	source-verified; output-invisible
H3 VF2Layout determinism (#14730)	fix_boundary	determinism	unreproduced (needs noisy target)
H4 VF2PostLayout no-op (#14120)	forward_regression	performance	confirmed (Stage-2 multi-run; up to ~340x at 27q)
H5 VF2Layout panic (#16285)	fix_boundary	functional	blocked (not run)

Table 3. Historical-event audit (5 historical events; H1 and H2 are source-verified through matched oracles — never through the black-box output oracle, which is blind to them by construction; the global-phase source-verification #14956 is reported in §6.4 / Table 5).

ID (PR)	Fault type → channel	candidate→baseline (short SHA)	Status	Reason
H1 (#14603)	semantic → contract/metadata	7c3890da→96fda188	real; output-invisible (source-verified)	corrupts virtual_permutation_layout, not the output unitary → invisible to the output oracle, caught by the contract oracle
H2 (#14919)	semantic →	14df5941→dfcc5c6c	real;	MR-1 violated on the

	contract/metadata		output-invisible (source-verified)	parent for all three routing passes; output correct up to permutation → output oracle blind
H3 (#14730)	quality → determinism	056c6413→d33ef533	blocked	non-determinism tied to a noisy Target absent from our coupling-map backend
H4 (#14120)	performance → performance	a18e1516→a8d23667	confirmed (Stage-2)	width-scaling slowdown 1.9x (16q) → ~340x (27q), Cliff's δ = 1.0; a verified performance forward-regression
H5 (#16285)	functional → compilation_failure	f0fae6f3→0b2cdf2b	not run	input-specific crash; not yet reproduced

6.2 Observability of historical regressions

We attempted five historical events, and we must separate a demonstrated observability gap from cases that are merely unreproduced or under-powered at this scale. Building the fixed and parent (or introducing) revisions from source and running the matched oracle, we establish three real transpiler regressions invisible to a black-box output oracle across two channels: contract/metadata (ElidePermutations #14603, final_layout composition #14919) and global phase (a CommutativeCancellation phase fix #14956, with a companion regression #16215 traced to its introducing commit by automated bisection). The determinism channel remains unreproduced at this scale; the small-overhead (performance) channel is separately confirmed as a forward-regression (H4, below). We report the determinism case as inconclusive rather than as evidence that the oracles under-detect.

- **H1 (ElidePermutations, #14603): demonstrated observability gap.** Both revisions were built from source. The fault belongs to the transpiler_contract_or_metadata channel: it corrupts the virtual_permutation_layout property while leaving the layout-applied output unitary correct. As a result, the output-only semantic oracle reports equivalence on the buggy build, even when the fix's own regression circuit is run through the full preset pass manager. The fault is therefore real but output-invisible: a black-box output oracle cannot see it, while the contract oracle detects it. This is one of three source-verified output-invisible regressions (with H2 #14919 and the global-phase fix #14956; §6.4 below), and Appendix A gives the minimal trigger, the exact SHAs and the corrupted field, the oracle outcome, and the reproduction command.
- **H3 (VF2Layout non-determinism, #14730): inconclusive (not reproduced).** The determinism-channel fault reproduces only under a noisy device target absent from our coupling-map backend; a strengthened, layout-sensitive determinism oracle observed determinism on our circuit. We cannot conclude anything about oracle sensitivity because the triggering condition was never created. Blocked. This is consistent with a large dynamic study that executed the Qiskit Terra suite 10,000 times across 23 releases and found genuine flakiness rare but often requiring thousands of runs to detect reliably, well beyond a typical CI budget [25].
- **H4 (VF2PostLayout no-op, #14120): confirmed performance forward-regression.** Single-sample Stage-1 timing left the redundant-pass overhead below the 20% screen, but a pre-declared multi-run Stage-2 protocol (7 repeats per unit, 2000-sample bootstrap CI, α = 0.05) confirms a width-scaling slowdown that grows from about 1.9x at 16 qubits to about 97x at 20 and about 342x at 27 (GHZ on a heavy-hex map, optimization level 3), with Cliff's δ = 1.0 (perfect separation) at every width. At that magnitude the effect is far beyond commodity-hardware timing noise, though a controlled-hardware replication remains future work. This is a fourth verified forward-regression event, in the observable performance channel.

- **H2 (final_layout, #14919): source-verified.** Building both revisions, the metamorphic relation MR-1 holds on the fixed build and is violated on the parent for all three routing passes, while the output oracle stays blind (§6.4). H5 (#16285) is an input-specific crash, not yet reproduced.

We therefore answer the observability question narrowly. We demonstrate three concrete observability gaps across two channels — contract/metadata (#14603, #14919) and global phase (#14956, with #16215 traced to its introducing commit) — each invisible to black-box output oracles, and it leaves the determinism channel open (unreproduced on commodity hardware), while the small-overhead channel is confirmed as a performance forward-regression (H4). We do not claim that black-box oracles broadly under-detect; we claim, and now demonstrate on real fixes, that at least two important manifestation channels are unobservable to them. Section 6.3 then quantifies how often such channels occur among real transpiler fixes.

6.3 Observability of real transpiler regressions: a repository-mining study

The H1 result raises a question the five historical events cannot answer on their own: how common are output-invisible regressions in practice? To estimate this, we mined merged bug-fix pull requests in the Qiskit repository and classified each one by the manifestation channel of the fault it fixes (Section 2.2). Label-based discovery alone is incomplete: a whole-library scan of one year of merged fixes found that the mod: transpiler and Changelog: Fixed labels surface only 28 of 39 distinct transpiler bug-fixes we could identify, and under-represent the global-phase channel the most (labels alone caught 3 of the 9 global-phase fixes). We therefore discovered candidates in three widening passes — (i) the mod: transpiler changelog label, (ii) a title and keyword sweep over pass names and channel terms, which recovers fixes tagged mod: circuit or left unlabelled (for example Commuting2qGateRouter and UnrollForLoops), and (iii) a semantic review of the remainder, which recovers cases such as a second CommutativeCancellation global-phase regression — so that the corpus is not biased toward label-visible faults, and we report the resulting count as a lower bound rather than a census. This follows repository-mining practice for quantum-software bug characterization [26] and the empirical-reporting guidance for quantum-software testing [24]. Three authors independently classified the fixes from their titles, descriptions, and linked issues, using a written, frozen codebook and recording a confidence flag for each. On a triple-coded, adjudicated seed of 24 fixes, agreement was substantial: Cohen's $\kappa = 0.67$ on the binary observable-by-output-oracle judgment (83.3% raw agreement) and $\kappa = 0.62$ on the seven-class channel (70.8%); disagreements were adjudicated against the frozen codebook, and in particular the ElidePermutations case (PR #14603, our H1) was confirmed output-invisible on the strength of direct built-from-source evidence. We then expanded the corpus to 68 in-scope merged transpiler bug-fixes (the 24-fix adjudicated seed plus 44 further fixes), which two authors coded independently, blind to each other's labels, against the frozen codebook (the coders did not confer during coding, and the seed adjudication was not revisited), reaching Cohen's $\kappa = 0.86$ on the binary observability judgment (95.5% raw agreement). The ten channel disagreements — only two of which touched the binary judgment — were adjudicated against the codebook, and both binary-relevant cases (PRs #14765 and #15074, edge-of-scope plumbing) resolved to the observable side. The codebook, all raters' labels, and the confidence flags are released with the artifact. To check that these labels track the actual fault rather than only its title, we validated a subsample of eleven fixes against source at two tiers of evidence: Tier 1, three cases built from source on which the fault is triggered and caught by the matched oracle while the output oracle stays blind (#14603, #14919, #14956); and Tier 2, eight further cases confirmed by inspecting the fix commit and its own regression test (#16201, #16215, #16402 and #15024 in the global-phase and contract/metadata channels, #14730, #15040, #16237 in the determinism channel, and the boundary fix #14765). The per-fix tier is recorded in label_source_validation.csv. All eleven matched the coded channel — ten of the nineteen output-invisible fixes plus one fix correctly coded observable (#14765) — with no mislabel, and the per-fix evidence is released as label_source_validation.csv. We complemented this with a symmetric false-negative audit of five observable-labeled fixes (#13670, #16337, #15626, #15967, #14998), each confirmed genuinely output-visible: #13670 and #16337 assert on commutation and operator-equality results, #15626 on a transpilation that previously raised, and #15967 on an instruction-count quality metric, while #14998 — flagged because its test inspects final_index_layout — was built from source and its buggy parent raises inside VF2PostLayout (a visible compilation failure) rather than silently mis-

recording the layout. Across all sixteen source-checked fixes no coded label was overturned. As Table 4 shows, 19 of the 68 classified fixes (≈ 28%; 95% Wilson CI 19–40%) fall in channels that a black-box output-equivalence oracle cannot observe: corrupted layout or permutation contract metadata (10), non-determinism (4), and dropped global phase (5, which are invisible to equivalence-modulo-global-phase checks). The built-from-source H1 case is one instance of the largest of these classes. The observability gap is therefore not an isolated curiosity. A substantial minority of real transpiler regressions are unobservable to output-only oracles, which motivates fault-class-matched oracles for this channel, applied as targeted per-fix and cross-version regression checks rather than relying on output equivalence alone.

Table 4. Manifestation channels of the 68 classified Qiskit transpiler bug-fixes. "Observable" = detectable by a black-box output-equivalence oracle. The 24-fix seed is triple-coded and adjudicated; the 44 further fixes are dual-coded (Cohen's κ = 0.86) and adjudicated against the same frozen codebook.

Manifestation channel	Count	Observable?	Example PRs
compilation_failure	24	yes	#13820, #14667, #14998, #15258, #15626
output_semantic	20	yes	#13670, #13790, #13874, #16337, #16428
contract / metadata	10	no	#13833, #13910, #13945, #14041, #14603
circuit_quality	4	yes	#14405, #14869, #15131, #15967
global phase	5	no	#14956, #15943, #16201, #16215, #16402
determinism / reproducibility	4	no	#14730, #14763, #15040, #16237
performance	1	yes (if over screen)	#16476

The seed (n = 24) is triple-coded and adjudicated (Cohen's κ = 0.67 binary, 0.62 channel); the 44 further fixes are dual-coded independently and blind (Cohen's κ = 0.86 binary, 95.5% agreement) and adjudicated against the same frozen codebook. All labels and confidence flags are released for re-coding. Classification is title-and-description based, a construct-validity threat we mitigate by releasing the codebook, all raters' labels, and the confidence flags for re-coding and by validating sixteen labels directly against source, in both directions (§6.3, label_source_validation.csv); the sample is recent merged fixes, not a complete census, so the percentages are estimates with the reported 95% Wilson CI rather than population parameters.

6.4 A fault-class-matched oracle family, and source-verified detection

The mining gap motivates oracles matched to each output-invisible channel. We implement three, all layout-aware and width-tiered so that they respect a hard cost boundary: an exact operator only for small circuits (n <= 12 qubits), sampled layout-normalised statevectors for 13-22 qubits, and a structural fallback beyond, never a full 2^n x 2^n unitary at scale. A contract differ asserts the transpiler's layout and permutation contract: the initial and final index layouts must be valid permutations, and they must compose consistently with the routing permutation. A metamorphic oracle encodes contract-level metamorphic relations; the first, MR-1 (permutation consistency), requires that appending the recorded routing_permutation to the routed output recover the input unitary. A global-phase oracle is the exact complement of the output-equivalence oracle: where the latter compares modulo global phase, the former measures the phase itself, so a dropped or double-counted global phase is detected end to end.

We verify three of the mined output-invisible fixes directly from source, building the fixed and parent revisions of each and running the matched oracle. For H1 (ElidePermutations, PR #14603) the contract oracle detects the fault as an asymmetric divergence, the buggy pass raising an error on the trigger where the fixed pass runs clean, while the full-pipeline output oracle passes the buggy build, which is exactly the observability gap. For the final_layout composition fix (PR #14919) the metamorphic relation MR-1 holds on the fixed build and is violated on the parent for all three routing

passes (SabreSwap, LookaheadSwap, BasicSwap), even though the routed circuit still implements the intended operation up to a qubit permutation, so an output-equivalence oracle sees nothing wrong. For the CommutativeCancellation global-phase fix (PR #14956) the global-phase oracle measures the π/2 phase that the fixed build introduces when merging an X into an X-rotation and that the parent drops to zero, while the output oracle — comparing modulo global phase — again sees nothing. In all three cases the fault is real, reachable, and invisible to output equivalence, and the matched oracle recovers it with no false positive on the fixed build. These three source-verified fixes establish that the faults are detectable on real regressions. To see how the oracles behave on faults they were not designed against, we complement them with a held-out evaluation on the mutation engine, whose injected faults are disjoint from the three design cases. Across 54 held-out runs (ten mutation families over six circuits) the matched oracles raised no false positives on the clean baselines (contract 0/54, global-phase 0/54, and MR-1 0/27 within its permutation-free precondition) and no spurious firing on these out-of-channel faults (0/54), so the family is channel-specific rather than trigger-happy; the incomplete-basis mutant instead surfaces as a visible compilation failure. Sensitivity is confirmed on a held-out synthetic global-phase corruption, which the phase oracle detects on all six circuits while the output-equivalence oracle stays blind on all six. The evaluation also surfaced a precondition we now enforce: MR-1 assumes the circuit carries no inherent qubit permutation, so it does not apply to circuits such as QFT with a built-in bit-reversal. The runner is released as scripts/heldout_oracle_eval.py.

Table 5. Fault-class-matched oracles and the mined output-invisible fixes verified from source.

Channel	Oracle	Source-verified fix	Output oracle
contract / metadata	contract differ + isolated-pass differential	#14603 ElidePermutations	blind
contract / metadata	metamorphic MR-1 (permutation consistency)	#14919 final_layout composition	blind
global phase	global-phase tracking (phase complement)	#14956 CommutativeCancellation	blind
determinism	fixed-seed metadata reproducibility	VF2 layout non-determinism (live)	blind

The differ asserts metadata-internal invariants — that the recorded index layouts are valid permutations and compose consistently with the routing permutation — and delegates full output equivalence to the semantic oracle; an earlier operator-level reconciliation check was removed after it raised false positives under the default VF2 post-layout while the layout-applied output was in fact correct. Swept across 243 circuit configurations (random and structured circuits, widths 4-8, line, ring and heavy-hex-like couplings, optimization levels 1-3), it reports no violation with the layout method fixed. Under the default layout path its metadata-reproducibility check surfaces a genuine determinism-channel artifact: fixed-seed transpilation returns different recorded layout metadata across identical calls, present on both Qiskit 2.4.2 and 2.5.0 and reproducible even single-threaded, which we trace to the time-budgeted VF2 layout search and report as a determinism-channel demonstration, not a new defect. The oracle surfaces this automatically; we report it as a demonstration of the oracle class rather than a new-defect claim, pending upstream triage.

6.5 Cross-SDK replication: is the gap Qiskit-specific?

To test whether the observability gap is specific to Qiskit, we replicated the mining protocol on two other quantum compilers with the SAME frozen codebook; the harvester is released as scripts/mine_sdk.py (a build-free clone-and-classify over each project's compilation-pass directories). Table 6 summarizes. On tket, a compiler architecturally close to Qiskit that exposes explicit placement, routing and permutation contracts, the output-invisible rate is 33% (7 of 21 in-scope bug-fixes; 95% Wilson CI 17–55%; Cohen's κ 0.77), comparable to Qiskit's 28% and

dominated by the SAME permutation/contract-metadata channel. The tket cases are direct analogues of ours: GreedyPauliSimp ignoring the circuit's implicit qubit permutation (#2072) and missing wire-swap handling in phase-polynomial synthesis (#146) mirror our ElidePermutations (H1) and final_layout (14919) faults.

On Cirq, whose leaner moment-based model exposes fewer layout/permutation-metadata contracts, the output-invisible rate is 20% (2 of 10 in-scope; Cohen's κ 0.52; the transformer commit history is otherwise dominated by non-behavioural housekeeping, which the scope gate excludes). The gap therefore replicates on both other compilers, bracketing Qiskit's 28% rather than fading. The tket and Cirq samples are small (n = 21, n = 10) with wide, overlapping Wilson intervals, and Cirq's inter-rater agreement is only moderate (Cohen's κ 0.52), so we read the result as a replication of the channel across compilers rather than as a precise ordering; any apparent trend with how much layout/permutation metadata a compiler exposes is at most a hypothesis for the larger study. The channel itself is real and cross-compiler. On the strength of this evidence we position the fault-class-matched oracles as targeted, per-fix and cross-version regression checks in the output-invisible channel rather than as a blanket addition to the compilation test suite. The equivalence-based tiers — the metamorphic and global-phase oracles — cannot build a dense operator or a statevector at the circuit widths transpiler tests routinely exercise, so at those widths only the structural and metadata-internal invariants apply; a suite-wide guarantee would additionally require evidence of widespread latent violations beyond the individual fixes already verified here, which the present corpus does not establish. We therefore leave the question of standing, suite-wide adoption to future work.

Table 6. Cross-SDK replication of the output-invisible rate. All three corpora are independently human-coded and adjudicated against the same frozen codebook; tket and Cirq are dual-coded (Cohen's κ 0.77 and 0.52) on small samples (n = 21, n = 10).

Compiler	Invisible / in-scope	Rate	95% Wilson CI	Dominant invisible channel
Qiskit	19 / 68	28%	19–40%	contract / permutation metadata
tket	7 / 21	33%	17–55%	contract / permutation metadata
Cirq	2 / 10	20%	6–51%	contract / permutation metadata

6.6 Are output-invisible fixes distinguishable by surface characteristics?

Returning to the 68-fix corpus, we ask whether output-invisible fixes can be spotted by cheap surface signals rather than by a matched oracle. For every fix we retrieved its GitHub metadata — lines added and deleted, files changed, commit count, and open-to-merge latency — and compared the nineteen output-invisible fixes against the forty-nine observable ones with a two-sided Mann–Whitney U test. On every dimension the two groups are statistically indistinguishable (Table 7): the medians nearly coincide and no comparison approaches significance (all $p \geq 0.12$, all Cliff's $|\delta| \leq 0.22$). Output-invisible fixes are therefore ordinary-looking — neither smaller, simpler, nor faster-merged than the fixes an output oracle already catches. This reinforces the central result: because these fixes cannot be separated from the rest by size, churn, or review latency, they cannot be triaged by a superficial heuristic, so a fault-class-matched oracle is necessary rather than merely convenient. The per-fix metadata and this summary are released (data/mining_validation/pr_characterization_raw.csv and pr_characterization_summary.csv).

Table 7. Output-invisible vs observable fixes: size, complexity, and merge latency (group medians; two-sided Mann–Whitney U, n = 19 vs 49). No dimension distinguishes the groups.

Metric	Invisible median	Observable median	U p	Cliff δ
lines added	52	54	0.66	+0.07

lines deleted	3	4	0.61	-0.08
files changed	3	3	0.12	+0.22
commits	2	2	0.53	-0.10
time-to-merge (days)	1.18	1.15	0.74	-0.05

7. Discussion

The central finding is an observability gap. A real transpiler regression, H1, lives in the transpiler_contract_or_metadata channel: its layout metadata is incorrect, yet the output unitary it produces is intact, and so a black-box output oracle cannot see it. We are careful not to over-generalize from a handful of events. The determinism channel (H3) remains open in our data because we could not reproduce it, not because we watched the oracles fail; the small-overhead channel (H4) is confirmed as a performance forward-regression (§6.2). The honest reading is therefore a caution rather than a sweeping claim: equivalence checking of compiled outputs is provably insufficient for at least two important classes of compiler regression (contract/metadata and global phase), and the contract/metadata, determinism, and performance channels each plausibly need an oracle of their own. The repository-mining study of §6.3 shows that these output-invisible channels account for roughly 28% of recent real transpiler bug-fixes (19 of 68; 95% Wilson CI 19–40%), which makes the gap quantitatively material rather than anecdotal. The methodology encodes the design consequences: test provenance is accounted for strictly, since a post-fix test can show that a fault is real but cannot prove it would have been caught in live CI, and fault-class-matched oracles are treated as first-class evidence.

Implications for practitioners. Because these faults pass the output-equivalence oracle by construction, a green output check is not sufficient evidence that a transpiler fix in the contract/metadata, global-phase, or determinism channel is correct. The practical consequence is that such a fix should ship with a targeted assertion on the recorded metadata — the layout/permutation contract, the tracked global phase, or fixed-seed reproducibility — alongside the usual output check. The released oracle family (the contract differ, the metamorphic MR-1 relation, and the global-phase tracker) supplies reusable per-fix and cross-version checks for exactly these channels, so the marginal cost of covering an output-invisible fix is small.

8. Threats to Validity

Construct. The largest threat is oracle observability (§7). A black-box oracle can miss contract/metadata-channel faults, as we demonstrate for H1, and it may also miss determinism- and performance-channel faults, which we did not test here, so a “pass” does not certify that a circuit is fault-free. We mitigate this by separating the cohorts, by using an empirical, confirmation-based false-positive rate, and by recording which oracle tier produced each label and which manifestation channel each historical fault occupies. The provenance rule also prevents post-fix tests from inflating the Primary claim. A further construct threat is mining completeness: label-based discovery misses unlabelled fixes, so we use the three-pass protocol of §6.3 (labels, keyword sweep, semantic review), validated against a whole-library scan, and report the 68-fix corpus as a floor rather than a census.

Internal. Timing on a single laptop is noisy, so we calibrate cost with CPU process time, treat performance as exploratory, and exclude high-load runs. Write-once raw artifacts, fixed seeds, and labels regenerated from the raw evidence all support reproducibility. Test non-determinism is a known confound in regression testing [27]. We therefore treat determinism and reproducibility as their own manifestation channel rather than folding them into correctness, and, consistent with the evidence that flaky tests are most detectable when newly added [28], we confine post-fix targeted triggers to the Secondary evaluation.

External. The corpus is small and specific to the Qiskit 2.x transpiler, so the results are scoped to that setting and the mined corpus; broader generalization needs the larger corpus described in §9. The verified forward-regression events are few (H1, #14919, and #16215 in the output-invisible

channels, plus the H4 performance event), so they may not represent the full fault distribution, and the mining classification is title-and-description based, a construct threat mitigated by releasing the codebook and all raters' labels and by the sixteen-fix, two-directional source validation of the labels (§6.3), which matched the coded channel in every case. A noisy-target determinism oracle, a broader sampled-statevector campaign, and a controlled-hardware performance protocol remain deferred to the scale-up.

Conclusion. With so few verified forward-regression events, we avoid significance claims and report per-event outcomes, binding any comparative claim to the rule that requires at least three verified forward-regression events. The leakage-absence gate guards against optimistic bias.

9. Limitations and Future Work

This study is bounded in scope. The observability finding rests on 68 human-coded fixes, with cross-SDK replication on small tket and Cirq samples and three source-verified detections, and the determinism channel could not be reproduced on commodity hardware. A planned scale-up will do three things. First, it will broaden the mined corpus and the cross-SDK samples so the rate is estimated more precisely and any cross-compiler ordering can be tested rather than merely hypothesized. Second, it will extend the fault-class-matched oracle family already built and source-validated here (the contract differ, MR-1, and the global-phase tracker; §6.4) with a noisy-target determinism oracle and a controlled-hardware performance protocol. Third, it will extend the introducing-commit tracing already applied to #16215 and #11399→#14919 to the remaining fix-boundary events, converting them into verified forward regressions. A companion study takes up the leakage-safe, cost-aware selection of these tests under budget, which the present paper deliberately leaves aside.

10. Conclusion

We asked how often real quantum-transpiler regressions escape the output-equivalence oracle that is meant to catch them, and what it takes to detect the ones that do. We mined 68 merged Qiskit transpiler bug-fixes under a frozen, human-adjudicated codebook, built a fault-class-matched oracle family for the output-invisible channels, and verified the gap from source, all within a leakage-safe evaluation methodology guarded by six validity gates and a strict claim-scope rule.

Building real transpiler fixes from source produced three clear positive findings across two channels: an ElidePermutations regression (#14603) and a final_layout composition regression (#14919) whose corrupted layout metadata is invisible to output-equivalence oracles, and a CommutativeCancellation regression (#14956) whose dropped global phase is invisible to equivalence-modulo-global-phase checks. The determinism channel could not be reproduced on commodity hardware and we report it as inconclusive rather than as evidence of oracle weakness; the small-overhead channel (H4) is confirmed as a performance forward-regression.

Taken together, the study contributes a quantified, cross-compiler, source-verified demonstration that equivalence checking of compiled outputs is an incomplete correctness criterion for the transpiler, together with the fault-class-matched oracle family that closes the gap and a rigorous, reproducible methodology. It also sets out a pre-specified path for a larger empirical evaluation using verified forward regressions and fault-class-matched oracles.

11. Artifact Availability

To support review and reproduction, the artifact is hosted in the public repository https://github.com/furqan-nr/quantum-observability (to be archived on Zenodo with a DOI upon acceptance), containing: the prototype (cart) and CLI; the automated test suite and six validity gates; the static manifest and mutation engine; the event ledger with exact candidate/baseline commit SHAs; the frozen change–stage map, pre-declared thresholds, and RNG seeds; a pinned environment lock (requirements.lock); the write-once raw oracle artifacts and derived labels; the generated evaluation.json; the repository-mining dataset (data/observability_mining.csv); and an OSI-approved open-source license.

The mining headline and the source-verified detections are reproduced as follows:

```
python scripts/score_worksheet.py
data/mining_validation/human_worksheet_44_R1.csv --rater2
data/mining_validation/human_worksheet_44_R2.csv
python scripts/verify_h1_isolated.py   # then verify_14919_routing.py,
source_validate_mining.py --only 14956
```

These reproduce the 19/68 headline (with κ) and the three source-verified detections. Building the historical events (e.g., H1) requires a Rust toolchain to compile the per-event Qiskit revisions from source; the per-event recipe and reproduction command are given in Appendix A.

Statements and Declarations

Funding. The authors received no specific grant from any funding agency, commercial, or not-for-profit sector for this work.

Competing Interests. The authors declare no competing financial or non-financial interests.

Data Availability. All data supporting the results, namely the change-event ledger, the write-once raw oracle artifacts, the derived labels, and the generated evaluation reports, are available in the public repository (https://github.com/furqan-nr/quantum-observability), to be archived on Zenodo with a DOI upon acceptance; see Section 11.

Code Availability. The cart prototype, its command-line interface, and the build scripts are available in the public repository (https://github.com/furqan-nr/quantum-observability, to be archived on Zenodo with a DOI upon acceptance) under the MIT open-source license; see Section 11.

Author Contributions. F. Nasir conceived the study, implemented the prototype, conducted the evaluation, and wrote the manuscript. A. Shah supervised the work and contributed to the study design and the critical revision of the manuscript. I. Alam contributed to the methodology and the analysis and reviewed the manuscript. All authors read and approved the final manuscript.

Ethics Approval and Consent. Not applicable; the study involves no human or animal subjects.

Use of AI Tools. AI-based assistants were used to support manuscript drafting and software implementation under the authors' supervision; the authors verified all results and take full responsibility for the content.

Appendix A. Direct evidence for H1 (ElidePermutations, PR #14603)

H1 is one of three source-verified observability gaps, and we record its evidence explicitly here.

- **Revisions (built from source).** Candidate (buggy) 7c3890da097c851876b86453a1da6ee7d3208048 → baseline (fixed) 96fda18888de4492f37ccdb93faf15dc014c99eb; fix-boundary differential, PR #14603, fix-commit dated 2025-06-16.
- **Minimal trigger.** A circuit containing a PermutationGate together with two-qubit gates (the unit trig-h1-elide-permutations, drawn from the fix's own regression test), transpiled through the full preset pass manager at optimization level 3 on backends none and line.
- **Corrupted field.** The buggy ElidePermutations pass corrupts the virtual_permutation_layout property recorded in the TranspileLayout / output metadata; the layout-applied output unitary is left correct.
- **Oracle outcome.** The layout-normalized exact-unitary oracle reports EQUIVALENT (pass) on the buggy candidate, identical to the baseline result, on both backends. The output-only oracle therefore cannot distinguish buggy from fixed; the fault is only visible by comparing the recorded permutation property directly (the contract/metadata check of §6.4). The output-only oracle is thus blind to this fault; it is real and output-invisible — a source-verified instance of the observability gap.
- **Reproduction.** Build both revisions with the per-event from-source runner, then run:

python -m cart.cli historical-run --event hist-elide-permutations-mapping --use-targeted

The recorded per-run property values (baseline vs candidate virtual_permutation_layout) are written to the event's write-once raw artifact and are included in the deposited archive (Section 11).

References

[1] Paltenghi M, Pradel M (2022) Bugs in quantum computing platforms: an empirical study. Proc ACM Program Lang (OOPSLA). arXiv:2110.14560

[2] Nation PD et al (2024) Benchmarking the performance of quantum computing software for quantum circuit creation, manipulation and compilation. Nat Comput Sci

[3] Paltenghi M, Pradel M (2023) MorphQ: metamorphic testing of the Qiskit quantum computing platform. In: Proc ICSE. doi:10.1109/ICSE48619.2023.00202

[4] Kitt L et al (2024) MorphQ++: a reproducibility study of metamorphic testing on quantum compilers. In: Proc IEEE/ACM ASE Workshops. doi:10.1145/3695750.3695823

[5] Pontolillo G et al (2025) QuCheck: a property-based testing framework for quantum programs in Qiskit. arXiv:2503.22641

[6] Luo J, Xia S, Zhang F, Zhao J (2026) QEMI: a quantum software stacks testing framework via equivalence modulo inputs. In: Proc FASE. arXiv:2602.09942

[7] Javadi-Abhari A, Treinish M, Krsulich K et al (2024) Quantum computing with Qiskit. arXiv:2405.08810

[8] Paltenghi M, Pradel M (2024a) A survey on testing and analysis of quantum software. arXiv:2410.00650

[9] Barr ET, Harman M, McMinn P, Shahbaz M, Yoo S (2015) The oracle problem in software testing: a survey. IEEE Trans Softw Eng 41(5):507–525

[10] Burgholzer L, Wille R (2021) Advanced equivalence checking for quantum circuits. IEEE Trans Comput-Aided Des Integr Circuits Syst 40(9):1810–1824

[11] Aparicio-Morales A, Moguel E, Garcia-Alonso J, Murillo JM (2024) An overview of quantum software engineering in Latin America. Quantum Inf Process 23

[12] Huang Y et al (2024) Qubit mapping: the adaptive divide-and-conquer approach. Quantum Inf Process 23

[13] Wang J, Zhang Q, Xu GH, Kim M (2021) QDiff: differential testing of quantum software stacks. In: Proc IEEE/ACM ASE, pp 692–704

[14] Ye J, Zhang F, Xia S, Guo X, Wu X, Zhao J, Xue Y (2026) QSPE: enumerating skeletal quantum programs for quantum library testing. arXiv:2602.00024

[15] Choudhury N et al (2025) Concolic testing for quantum compilers. In: Proc IEEE ICCD

[16] Tao R, Shi Y, Yao J, Li X, Javadi-Abhari A, Cross AW, Chong FT, Gu R (2022) Giallar: push-button verification for the Qiskit quantum compiler. In: Proc ACM SIGPLAN PLDI, pp 641–656

[17] Paltenghi M, Pradel M (2024b) Analyzing quantum programs with LintQ: a static analysis framework for Qiskit. Proc ACM Softw Eng 1(FSE):1135–1157

[18] Zhao P, Zhao J, Miao Z, Lan S (2023) Bugs4Q: a benchmark of existing bugs to enable controlled testing and debugging studies for quantum programs. J Syst Softw 205:111805

[19] Mendiluze E, Ali S, Yue T, Arcaini P (2021) Muskit: a mutation analysis tool for quantum software testing. In: Proc IEEE/ACM ASE

[20] Fortunato D, Campos J, Abreu R (2022) Mutation testing of quantum programs: a case study with Qiskit. IEEE Trans Quantum Eng 3:1–17

[21] Ramalho NCL et al (2024) Testing and debugging quantum programs: the road to 2030. ACM Trans Softw Eng Methodol

[22] Wojciechowski K et al (2026) Open QBench: a benchmarking framework for evaluating quantum computing platforms. Quantum Inf Process

[23] Mendiluze Usandizaga E, Yue T, Arcaini P, Ali S (2023) Quantum circuit mutants: empirical analysis and recommendations. Empir Softw Eng 28:6

[24] Li Y et al (2026) A methodological analysis of empirical studies in quantum software testing. ACM Trans Softw Eng Methodol. arXiv:2601.08367

[25] Kim D et al (2025) Detecting flakiness in quantum software: a dynamic testing approach. arXiv:2512.18088

[26] Yousuf MM, Sofi SA (2025) Characterizing bugs and quality attributes in quantum software: a large-scale empirical study. arXiv:2512.24656

[27] Luo Q, Hariri F, Eloussi L, Marinov D (2014) An empirical analysis of flaky tests. In: Proc ACM SIGSOFT FSE, pp 643–653

[28] Lam W, Winter S, Wei A, Xie T, Marinov D, Bell J (2020) A large-scale longitudinal study of flaky tests. Proc ACM Program Lang 4(OOPSLA):1–29